PEPPER: Shell Scripting In Python

Ridhi Srikanth, rsrikanth@ucsd.edu UC San Diego

Motivating Example

Python excels at computation and data processing, while the UNIX shell excels at filesystem interaction and command composition. Yet combining them often requires switching languages or using APIs like subprocess.

Here we want to process all JPG images by resizing them to 256×256, computing their average pixel intensity, and saving the sorted results to a CSV:

Defined Python Function:

```
import cv2

def process(paths):
    def transform(path):
        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        img = cv2.resize(img, (256, 256))
        return f"{path},{img.mean():.2f}"

    return "\n".join(transform(p) for p in paths)
```

Standard Python

```
import subprocess

data = subprocess.run(
    ["find", "images", "-name", "*.jpg"],
    capture_output=True, text=True, check=True
).stdout.splitlines()

result = subprocess.run(
    ["sort"], input=process(data),
    capture_output=True, text=True, check=True
).stdout

with open("processed.csv", "w") as f:
    f.write(result)
```

Need to explicitly convert data to a list.

Verbose process orchestration.

PEPPER

PEPPER automatically converts the shell text output to a list.

```
data = find images -name "*.jpg"
process(data) $| sort $> "processed.csv"
```

Preserves concise shell-like syntax for pipelines and redirection

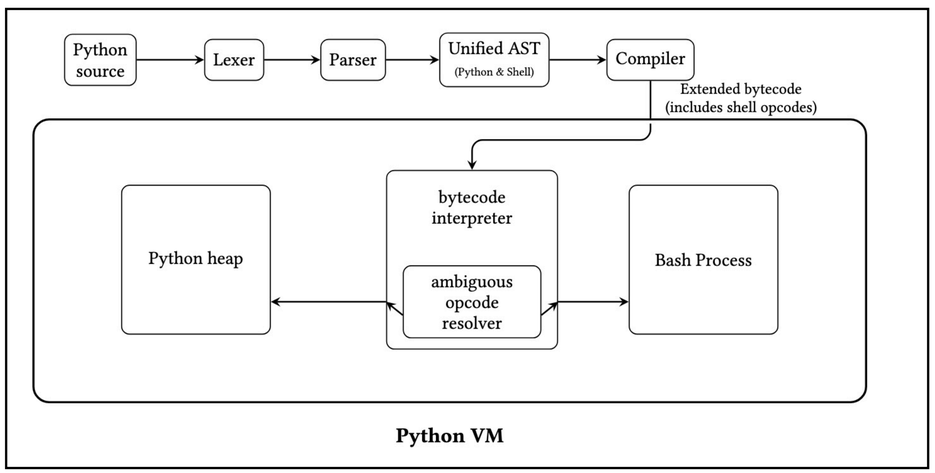
Shell

```
find images -name "*.jpg" | python process.py | sort > processed.csv
```

Requires writing an external Python script with custom stdin parsing.

PEPPER

PEPPER brings common shell constructs directly into Python, enabling Python computation and shell operations in one program while preserving existing Python syntax and libraries. This is done by extending the CPython interpreter.



Construct	Syntax	Example
Shell command	`<command> [<flag>] [<args>]`	`ls -l -color`
	flag: `-l, -color`	`grep "Alice" "file.txt"`
	shell arg: `"..."`	`grep name "file.txt"`
	Python var as arg: `<name>`	*where* `name` *is a Python variable*
Pipeline	`<command> $| <command>`	`cat "data.txt" $| grep "Alice"`
Output redirection	`<command> $> "<filename>"`	`echo "hello" $> "out.txt"`
Input redirection	`<command> $< "<filename>"`	`sort $< "data.txt"`
Append redirection	`<command> $» "<filename>"`	`echo "hello" $» "out.txt"`
Environment variable	`$<name>`	`echo $HOME`
Environment assignment	`$<name> = <expr>`	`$path = "/tmp"`

Evaluation

To what extent does PEPPER improve task completion and success rates:

- **for file-system manipulation tasks? (RQ1)**
- **when orchestrating multi-process data-processing pipelines? (RQ2)**

We are conducting a controlled user study to evaluate the usability and effectiveness of PEPPER. The data from 12 participants are shown below:

RQ3: To what extent does PEPPER maintain semantic compatibility with standard Python?

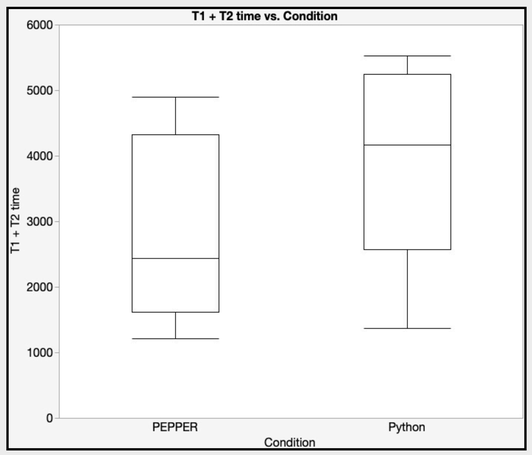
We ran the official CPython 3.13 regression test suite, covering Python language features and standard library behavior.

PEPPER passes 43,814/43,828 executed tests (99.96%) across 480 CPython 3.13 test modules. 435 modules pass, 32 skip, 3 are omitted, and 10 fail.

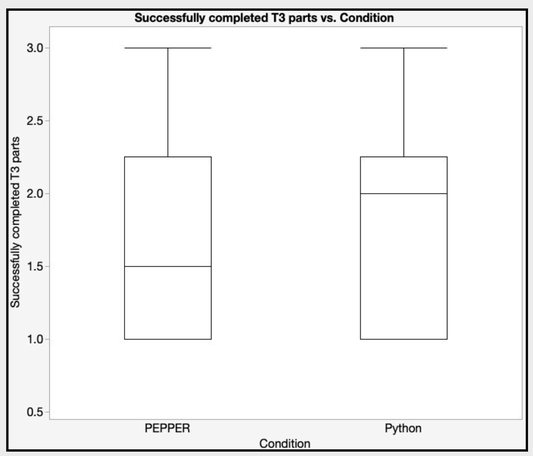
RQ4: To what extent does PEPPER replicate the behavior of real-world POSIX shell programs?

We selected 66 POSIX shell scripts from across the Koala benchmark, excluding scripts that use shell function definitions or other shell constructs not currently supported by PEPPER. We transpiled these scripts into PEPPER and compared their stdout and exit codes with native execution.

All 66 scripts produced identical outputs and exit codes.



Total time in seconds to complete filesystem and pipeline coding tasks (T1 + T2) across experimental conditions (N = 12).



Number of correctly completed tasks across conditions (N = 12). For the comprehension task (T3), success is defined as selecting the correct code option.

Efficiency (T1 + T2): PEPPER users finished substantially faster, most outperforming the standard Python median.

Comprehension (T3): Python yielded slightly higher typical accuracy, reflecting prior familiarity over PEPPER's novel syntax.

Status and Upcoming Work

- Core implementation, Python compatibility and behavioral equivalence with shell complete.
- Concluding user study and based on participant feedback we are implementing expression-level embedding.